

“青隅”校园互助服务平台 P1-P3 问题总结情况与修改建议文档

项目名称：青隅 CampusHub 校园互助服务平台

涉及阶段：P1 需求分析、P2 架构设计、P3 详细设计

文档用途：总结当前 P1-P3 阶段存在的问题，分析阶段之间的适配情况，并提出面向后续修改与 P4 编码落地的具体建议。

一、总体结论

当前项目已经完成 P1 需求分析、P2 架构设计和 P3 详细设计三个关键阶段，整体形成了较完整的软件工程前期设计链路：

P1：明确做什么 → P2：明确系统怎么组织 → P3：明确类、接口、数据库如何落地

从整体看，项目主线是清楚的：围绕高校学生的匿名倾诉、找搭子、恋爱互助等场景，构建一个校园互助服务平台。P1 提供了较丰富的业务场景，P2 给出了较务实的单体分层架构方案，P3 进一步补充了类图、API、数据库、安全设计等详细设计产物。

但是，当前 P1-P3 之间仍存在一些需要统一和修改的问题。主要表现为：

1. P1 需求覆盖较广，但 MVP 边界不够明确；
2. P2 架构选择务实，但对匿名、审核、通知、状态流等关键机制的说明仍可加强；
3. P3 详细设计较完整，但接口覆盖不均衡，状态枚举和命名规范需要统一；
4. P1 的部分需求、P2 的最简架构原则、P3 的完整详细设计之间存在复杂度不完全匹配的问题；
5. 如果不在 P4 前收敛开发范围，后续编码阶段可能出现“设计很完整，但实现压力过大”的问题。

因此，当前最重要的修改方向不是继续扩展功能，而是：**统一名称、收敛范围、补齐关键接口、统一状态枚举、明确 MVP 实现边界。**

二、P1 需求分析阶段问题总结与修改建议

2.1 P1 当前完成情况

P1 阶段已经完成了需求规格说明书、用户故事、用例描述、调研方案、访谈笔记等内容。整体上，P1 能够较清楚地描述项目目标用户、业务场景和主要功能模块。

P1 的优势主要包括：

1. 产品定位较明确，聚焦校园互助社交；
2. 用户故事贴近大学生真实生活场景；
3. 覆盖了树洞、找搭子、恋爱互助三大核心模块；
4. 考虑了匿名、隐私保护、内容审核等重要风险；
5. 除前台功能外，也考虑了管理员、审核、反馈等后台支撑模块。

2.2 P1 存在的主要问题

问题 1：需求范围偏大，MVP 边界不够清晰

P1 中同时包含树洞、搭子、恋爱互助、聊天、语音、心理健康引导、智能匹配、内容审核、消息通知、反馈建议等功能。虽然这些功能都符合产品方向，但对于 4 人团队和课程项目周期而言，范围偏大。

如果不明确哪些功能必须实现，哪些功能只是扩展设计，后续 P4 编码阶段容易出现任务过重的问题。

修改建议

在 P1 需求规格说明书中增加“版本范围与优先级说明”，将需求分为三层：

层级	功能范围	说明
Must 必须实现	注册登录、实名认证、树洞发布/浏览/评论/点赞、搭子发布/浏览/申请、管理员审核	支撑系统基本运行和课程展示
Should 应当实现	消息通知、举报处理、反馈提交、恋爱资料、基础心动匹配	增强系统完整性
Could 可选实现	实时聊天、语音消息、复杂推荐、人脸识别、第三方图片审核、心理健康专栏	作为后续扩展或展示亮点

建议在 P1 文档中明确写入：

本项目第一阶段以 MVP 核心功能为主，优先实现用户体系、树洞、找搭子、基础审核和通知能力；恋爱互助、实时聊天、语音、人脸识别等功能作为增强模块或后续扩展。

问题 2：部分需求仍偏“故事化”，可测试性不足

P1 中有些需求描述较贴近真实场景，但还没有完全转化为可测试、可实现的功能规则。例如：

- “保护隐私”没有完全说明哪些字段隐藏、哪些场景可见；
- “智能匹配”没有说明是条件筛选还是算法推荐；
- “内容审核”没有说明发布前审核、发布后审核还是人工复核；
- “恋爱互助”中的心动墙、匿名情感求助、双向心动之间的数据边界不够清楚。

修改建议

在 P1 中补充“可测试验收标准”，将抽象需求改为具体规则。

原需求表达	建议修改为
保护用户隐私	匹配前不返回手机号、学号、真实姓名；树洞列表不返回 userId；管理员审核场景可追溯 userId
智能匹配搭子	第一版采用条件筛选 + 用户主动申请，后续扩展算法推荐

原需求表达	建议修改为
内容审核	发布后进入 PENDING 状态，审核通过后变为 PUBLISHED，驳回后变为 REJECTED
双向心动	单向心动不通知对方，双方互相心动后状态变为 MUTUAL 并允许聊天

问题 3：项目名称和产品命名需要统一

当前项目中可能同时出现“校园互助服务平台”“CampusHub”“青隅”等名称。多名称并存本身可以接受，但需要明确主次关系，否则文档显得不统一。

修改建议

在 P1、P2、P3 所有文档开头统一写法：

项目中文名：青隅
项目英文名：CampusHub
项目定位：面向高校学生的校园互助服务平台

正文中建议优先使用“青隅”，第一次出现时写作：

“青隅”校园互助服务平台（英文名 CampusHub）

问题 4：用户角色和权限边界需要更明确

P1 已经涉及普通用户、管理员、审核员等角色，但权限边界仍可进一步明确。例如：

- 游客是否可以浏览树洞？
- 未实名认证用户能做什么？
- 被禁言用户能否浏览？能否点赞？
- 管理员是否能查看匿名发布者真实身份？
- 审核员和超级管理员权限是否不同？

修改建议

在 P1 中补充“用户角色权限表”：

角色/状态	可执行操作	不可执行操作
游客	浏览公开树洞	发布、评论、点赞、申请搭子
注册未认证用户	浏览公开内容、修改资料	发布树洞、发布搭子、恋爱匹配
实名认证用户	发布树洞、评论、搭子申请、心动匹配	管理员审核操作
禁言用户	浏览内容	发布、评论、申请、聊天
审核员	审核内容、驳回内容	修改系统配置

角色/状态	可执行操作	不可执行操作
管理员	用户管理、内容管理、审核记录查看	无授权的数据导出

三、P2 架构设计阶段问题总结与修改建议

3.1 P2 当前完成情况

P2 阶段已经完成了架构设计文档、ADR 集合、架构候选方案对比、架构辩论记录等内容。整体架构采用：

分层架构 + MVC 表现层 + 前后端分离 + RESTful API + JWT + MySQL 单库

这个选择总体是合理的，能够匹配 4 人团队、10 周周期和课程项目交付要求。

P2 的优势主要包括：

1. 架构选择务实，没有盲目采用微服务；
2. 技术栈清晰，Spring Boot + Vue 3 + MySQL 适合课程项目；
3. ADR 记录了关键技术取舍；
4. 明确暂不引入 Redis，避免过早优化；
5. 能够承接 P1 中的大部分功能模块。

3.2 P2 存在的主要问题

问题 1：匿名与隐私保护架构说明仍需加强

P1 很强调匿名树洞、匿名情感求助、匹配前隐私保护。P2 虽然选择了 JWT 和用户认证，但 JWT 只能解决“用户是谁”和“是否有权访问”，不能自动解决匿名展示和隐私隔离问题。

修改建议

在 P2 架构设计文档中增加“匿名与隐私保护设计”小节，建议写入：

系统内部所有用户行为仍绑定真实 user_id，用于安全审计、违规追溯和数据管理；前端展示层根据场景返回匿名昵称或脱敏字段，不直接返回真实姓名、手机号、学号等敏感信息。

并补充规则表：

场景	后端保存	前端展示	是否可追溯
树洞发布	user_id + anonymous_name	anonymous_name	管理员可追溯
树洞评论	user_id + anonymous_name	anonymous_name	管理员可追溯
搭子未匹配	user_id	昵称、年级、专业	普通用户不可见隐私
搭子已匹配	user_id	基础资料、可选联系方式	双方有限可见

场景	后端保存	前端展示	是否可追溯
审核后台	user_id、内容记录	审核员按权限查看	可追溯

问题 2：内容审核状态流需要在架构层统一

P2 中提到内容审核，但对内容状态流说明还不够完整。P3 已经设计了一些状态，但为了 P1-P3 一致，P2 中也应补充统一的审核流程。

修改建议

在 P2 中增加“内容审核流程设计”：

用户提交内容 → 自动敏感词检测 → 进入待审核状态 PENDING → 管理员/审核员审核 → 通过后 PUBLISHED → 驳回后 REJECTED → 被举报后 HIDDEN → 删除后 DELETED

推荐状态流：

状态	含义
DRAFT	草稿
PENDING	待审核
PUBLISHED	已发布
REJECTED	审核驳回
HIDDEN	被举报或管理员隐藏
DELETED	用户或管理员删除

问题 3：消息通知架构需要明确是否实时

P1 中存在申请提醒、审核提醒、评论提醒、系统通知等需求。P2 中有通知模块，但没有明确第一版是否使用 WebSocket、消息队列或轮询。

修改建议

根据课程项目复杂度，建议 P2 明确：

第一版采用数据库站内通知机制，不引入 WebSocket 和消息队列。用户通过接口轮询或页面刷新获取未读通知。后期如有实时性需求，可扩展 WebSocket 或消息队列。

这样既能满足需求，又不违背 P2 的最简架构原则。

问题 4：文件存储设计需要补充边界

P2 选择本地文件存储，这是课程项目中合理的简化方案。但系统涉及头像、树洞图片、搭子图片、恋爱照片、反馈截图、语音文件等，如果不说明边界，后续实现会混乱。

修改建议

在 P2 中补充本地文件存储策略：

/upload/avatar/	用户头像
/upload/treehole/	树洞图片
/upload/partner/	搭子图片
/upload/love/	恋爱交友照片
/upload/feedback/	反馈截图
/upload/voice/	语音文件，可选

并说明：

1. 文件路径存入数据库；
2. 文件大小在 API 层限制；
3. 删除内容时优先逻辑删除，文件可定期清理；
4. 后期可迁移到对象存储。

问题 5：P2 与 P3 的复杂度需要重新对齐

P2 强调最简架构和快速交付，但 P3 设计了较多高级能力，如语音、聊天、Refresh Token、人脸识别、第三方审核、观察者模式、策略模式等。

修改建议

在 P2 或 P3 整合文档中增加“架构实现边界说明”：

类型	当前阶段处理方式
微服务	不采用
Redis	暂不引入
消息队列	暂不引入
WebSocket	暂不实现，后期扩展
第三方短信	可先用模拟验证码
人脸识别	可作为设计预留，不进入 MVP
图片审核	可先用敏感词/人工审核替代

四、P3 详细设计阶段问题总结与修改建议

4.1 P3 当前完成情况

P3 已经新增类图设计、SOLID 检查、API 规范、ER 图与建表 SQL、详细设计整合版、AI 协作反思日志。整体完成度较高，已经具备进入 P4 编码阶段的基础。

P3 的优势主要包括：

1. 类图将 P2 模块进一步落成类和类关系；
2. API 文档统一了 `/api/v1`、JSON、JWT、分页、错误码；
3. 数据库设计给出了 18 张表和建表 SQL；
4. SOLID 检查体现了对 AI 输出的人工审查；
5. 安全设计对密码、Token、隐私、审核进行了补充。

4.2 P3 存在的主要问题

问题 1：API 覆盖不均衡

当前 API 文档中认证、树洞、搭子相关接口较完整，但恋爱互助、审核、通知、反馈接口仍不够完整。

修改建议

补充以下接口：

恋爱互助接口：

```
POST /api/v1/love/profiles
PUT  /api/v1/love/profiles/me
GET  /api/v1/love/profiles
POST /api/v1/love/requests
POST /api/v1/love/requests/{id}/heart
GET  /api/v1/love/matches
```

审核管理接口：

```
GET /api/v1/admin/reviews/pending
GET /api/v1/admin/reviews/{reviewId}
PUT /api/v1/admin/reviews/{reviewId}
POST /api/v1/admin/users/{userId}/ban
```

通知接口：

```
GET /api/v1/notifications
PUT /api/v1/notifications/{id}/read
PUT /api/v1/notifications/read-all
```

反馈接口：

```
POST /api/v1/feedback
GET /api/v1/feedback/me
GET /api/v1/admin/feedback
PUT /api/v1/admin/feedback/{id}
```

问题 2：状态枚举需要统一

P3 中不同模块存在多种状态表达，如：

```
PENDING / PUBLISHED / PASSED / APPROVED / REJECTED / CLOSED / EXPIRED / ENDED
```

这些状态含义存在交叉，如果不统一，编码阶段容易出现重复枚举和判断混乱。

修改建议

在 P3 详细设计整合版中增加“统一状态枚举表”：

对象类型	推荐状态
内容类	DRAFT / PENDING / PUBLISHED / REJECTED / HIDDEN / DELETED
申请匹配类	PENDING / ACCEPTED / REJECTED / CANCELED / ENDED
审核记录类	PASSED / REJECTED / WARNING
反馈类	PENDING / VIEWED / PROCESSING / RESOLVED / CLOSED
账号类	NORMAL / WARNED / MUTED / BANNED
认证类	UNCERTIFIED / PENDING / CERTIFIED / REJECTED

问题 3：数据库字段与 API 字段命名规则需要明确

数据库字段采用 snake_case，例如 created_at、user_id；API 返回使用 camelCase，例如 createdAt、userId。这种设计合理，但必须在 P3 中明确转换规则。

修改建议

在 API 文档或数据库文档中增加命名规范：

```
数据库字段统一使用 snake_case；
Java 实体类属性统一使用 lowerCamelCase；
API JSON 字段统一使用 lowerCamelCase；
DTO 层负责实体字段与接口字段之间的转换。
```

API 路径建议统一采用模块层级形式：

```
/api/v1/auth/register  
/api/v1/treehole/posts  
/api/v1/partner/requests  
/api/v1/love/profiles  
/api/v1/admin/reviews
```

避免混用 `/partner-requests` 与 `/partner/requests`。

问题 4：数据库 JSON 字段较多，需要说明取舍

P3 数据库中存在多个 JSON 字符串字段，如 `conditions`、`interests`、`mate_preference`、`images`、`photos`、`permissions`。这样可以降低建表复杂度，但会影响复杂查询。

修改建议

在数据库文档中增加说明：

课程项目第一版为了降低表结构复杂度，对图片列表、兴趣标签、择偶偏好、权限列表等低频复杂字段采用 JSON 字符串存储；对于高频查询字段，如年级、专业、性别、地点等，应优先拆分为独立字段或建立冗余索引字段。

建议短期保留：

- `images`
- `photos`
- `permissions`

建议考虑拆分或冗余：

- `partner conditions` 中的 `grade / major / gender / location`
- `love mate_preference` 中的核心筛选条件
- `user interests` 如果后期用于推荐，可拆为标签表

问题 5：P3 设计范围需要标明 MVP 与扩展

P3 中设计了聊天、评价、语音、第三方审核、人脸识别、Refresh Token、AES 加密等内容。设计层面合理，但 P4 不应全部实现。

修改建议

在 P3 详细设计文档中增加“P4 编码实现范围建议”：

类型	功能	P4 是否实现
核心功能	注册登录、JWT、树洞、搭子、审核	必须实现
支撑功能	通知、反馈、举报	优先实现基础版
增强功能	恋爱资料、心动匹配	视时间实现
扩展功能	实时聊天、语音、人脸识别、第三方审核	可暂缓

五、P1-P2-P3 阶段适配性总结

5.1 适配良好的部分

阶段链路	当前表现	评价
P1 用户需求 → P2 用户模块 → P3 用户表/API	注册、登录、认证设计完整	适配良好
P1 树洞需求 → P2 树洞模块 → P3 树洞类/表/API	匿名发布、评论、点赞均有设计	适配良好
P1 搭子需求 → P2 搭子模块 → P3 搭子类/表/API	发布、浏览、申请、评价较完整	适配良好
P1 审核需求 → P2 审核模块 → P3 审核表/策略	有审核记录和敏感词设计	基本适配
P1 隐私需求 → P2 JWT/匿名设计 → P3 加密/脱敏	已有承接，但规则需进一步细化	基本适配
P2 MySQL 单库 → P3 18 张表 SQL	数据库设计完整	适配良好
P2 RESTful → P3 API 规范	基础规范已落地	适配良好

5.2 适配不足的部分

问题	表现	修改方向
P1 需求过多，P3 设计过全	P4 实现压力大	明确 MVP
P1 恋爱互助较丰富，P3 API 不够完整	有表和类，但接口不足	补充 Love API
P1 管理员审核是核心用例，P3 API 不够突出	有审核表，但接口不足	补充 Admin Review API
P2 最简架构原则与 P3 扩展设计存在张力	P3 有较多复杂能力	标明扩展功能暂不实现
P2 本地文件存储，P3 多处文件字段	文件管理规则不足	补充上传/删除策略
P3 状态与命名不统一	后续编码易混乱	统一枚举和命名规范

六、优先修改清单

6.1 高优先级修改

序号	修改项	修改位置	原因
1	增加 MVP 范围说明	P1、P3	防止 P4 编码失控

序号	修改项	修改位置	原因
2	统一状态枚举	P2、P3	防止业务状态混乱
3	补充审核 API	P3 API 文档	管理员审核是核心流程
4	补充通知和反馈 API	P3 API 文档	数据表已有但接口缺失
5	统一项目命名	P1、P2、P3	提升文档一致性

6.2 中优先级修改

序号	修改项	修改位置	原因
1	补充恋爱互助 API	P3 API 文档	承接 P1 恋爱需求
2	补充匿名与隐私规则表	P2、P3	强化核心安全设计
3	补充字段命名规则	P3 API/数据库文档	方便前后端联调
4	补充本地文件存储策略	P2、P3	避免文件管理混乱
5	说明 JSON 字段取舍	P3 数据库文档	提升数据库设计说服力

6.3 低优先级修改

序号	修改项	修改位置	原因
1	优化复杂推荐说明	P1、P3	可作为后续扩展
2	细化人脸识别方案	P1、P3	不建议 MVP 实现
3	细化第三方图片审核方案	P2、P3	可用人工审核替代
4	补充 WebSocket 扩展说明	P2、P3	当前无需实现

七、建议修改后的阶段定位

7.1 P1 修改后定位

P1 应重点回答：

系统要解决什么问题？哪些功能第一版必须实现？不同角色有什么权限？每个功能如何验收？

修改后的 P1 应更加关注：

1. 需求优先级；
2. MVP 范围；
3. 用户角色权限；
4. 可测试验收标准；
5. 隐私保护规则。

7.2 P2 修改后定位

P2 应重点回答：

系统采用什么架构？为什么这样选？关键机制如何支撑需求？哪些复杂技术暂不采用？

修改后的 P2 应更加关注：

1. 匿名机制；
2. 审核状态流；
3. 通知机制是否实时；
4. 文件存储边界；
5. 简化架构与扩展设计之间的关系。

7.3 P3 修改后定位

P3 应重点回答：

系统具体有哪些类、接口、表？这些设计是否能直接指导编码？哪些进入 P4 实现？

修改后的 P3 应更加关注：

1. API 完整性；
2. 状态枚举统一；
3. 数据库与 API 字段映射；
4. MVP 编码范围；
5. 数据表与接口的一一对应关系。

八、P4 编码前建议确认事项

进入 P4 前，建议团队先开一次短会，确认以下问题：

1. P4 是否只实现 Web 端，Flutter 是否暂缓？
 2. 后端使用 MyBatis 还是 JPA？
 3. 18 张表是否全部建表，还是先建 MVP 相关表？
 4. 短信验证码是否真实接入，还是使用模拟验证码？
 5. 第三方图片审核是否真实接入，还是使用人工审核模拟？
 6. 管理员后台是否做页面，还是先做接口和 Postman 测试？
 7. 恋爱互助模块是否进入第一轮开发？
 8. 聊天功能是否真实实现，还是暂时只设计不编码？
 9. 是否必须做前端页面，还是以后端接口演示为主？
 10. 每个成员负责哪些模块？
-

九、最终总结

当前 P1、P2、P3 三个阶段整体完成度较高，已经形成了较完整的课程项目设计链路。P1 提供了较丰富的业务需求，P2 给出了合理的单体分层架构，P3 进一步完成了类图、API、数据库和安全设计。

现阶段最主要的问题不是设计缺失，而是设计需要进一步统一和收敛。尤其需要解决以下五件事：

1. **P1 收敛需求范围，明确 MVP；**
2. **P2 补充匿名、审核、通知、文件存储等关键机制；**
3. **P3 补齐恋爱、审核、通知、反馈 API；**
4. **统一 P1-P3 中的状态枚举、命名规范和字段映射；**
5. **在 P4 前明确哪些功能必须实现，哪些功能只作为扩展设计。**

如果完成以上修改，项目将从“文档完整”进一步提升为“设计可落地、编码可执行、答辩有逻辑”的状态。